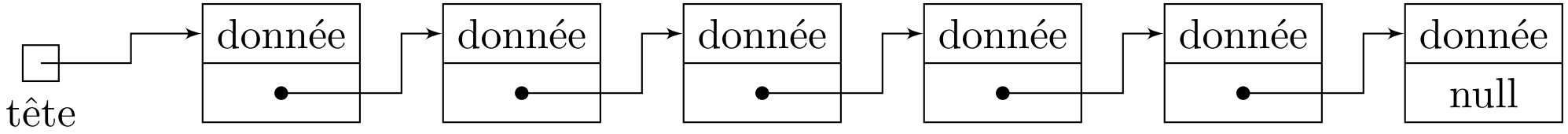
Listes chaînées

Définition d'une liste chaînée



- Une valeur
- Une référence vers la suite de la liste

Remarque : la suite de la liste est une liste chaînée $\longrightarrow$ définition récursive

Vocabulaire : un élément d'une liste sera aussi appelé **maillon** dans le cours

Remarque : la donnée peut être de n'importe quel type ; dans la suite du cours, on utilisera des nombres.

Remarque : à la différence des tableaux, il s'agit d'une structure non contiguë en mémoire

Opérations de base sur les listes chaînées

- Créer une liste vide
- Ajouter un élément en tête de liste
- Savoir si la liste est vide
- Obtenir la valeur stockée en tête de liste
- Obtenir la suite de la liste

Que faire avec ces opérations de base ?

Avec les opérations de base, on peut plus ou moins facilement :

- Accéder au i^e élément
- Ajouter un élément à une position quelconque
- Supprimer un élément

Comparaison tableau / liste chaînée

Tableau	Liste chaînée
Avantages	
Accès direct à un élément	Insertion / suppression plus simple (notamment en tête)
Inconvénients	
Insertion / suppression au milieu	Pas d'accès direct à un élément

Implémentation d'une liste chaînée

- Définition d'une structure de données pour représenter un maillon
- Une liste chaînée est une référence sur le maillon de tête
- Attention à l'implémentation des opérations
- Par exemple, pour l'ajout / suppression en tête, il faut penser à mettre à jour la référence représentant la liste

Une solution : encapsulation dans un objet

Structure `Maillon`

donnée
null

Cette structure doit contenir deux choses :

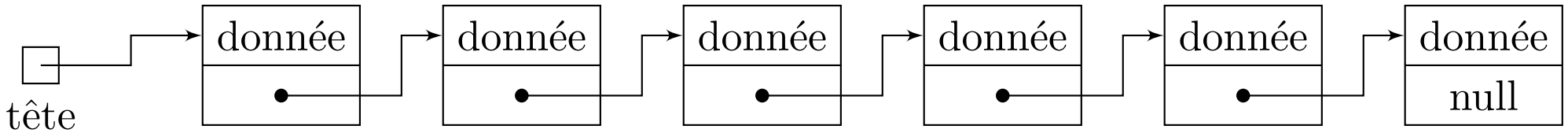
- la donnée stockée
- Une référence sur le prochain maillon

Remarque : Il faut choisir une valeur particulière (`null`) si le maillon ne pointe sur rien (par exemple, le dernier maillon)

Implémentation de `maillon` en TypeScript

```
class Maillon {
  private donnée_: number;
  private suivant_: Maillon | null; // référence sur un maillon ou valeur null

  constructor(donnée: number, suivant: Maillon | null) {
    this.donnée_ = donnée;
    this.suivant_ = suivant;
  }

  get donnée(): number {
    return this.donnée_;
  }

  set donnée(valeur: number) {
    this.donnée_ = valeur;
  }

  get suivant(): Maillon | null {
    return this.suivant_;
  }

  set suivant(valeur: Maillon | null) {
    this.suivant_ = valeur;
  }
}
```

Structure `ListeChaînée`



- Une référence sur le maillon de tête
- A la construction, une liste est vide dans ce cas le maillon doit pointer sur `null`

Remarque : pour les méthodes, il faut faire attention au cas particulier comme :

- ajout d'une liste vide
- retrait de l'élément de tête quand la liste ne contient qu'un élément

Il faut alors vérifier s'il n'y a pas un traitement particulier à faire.

Implémentaton en TypeScript

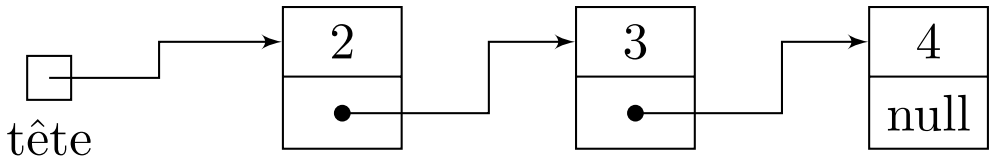
```
class ListeChaînée {
  private tête_: Maillon | null = null; // Initialement la liste est vide

  get tête(): Maillon | null {
    return this.tête_;
  }

  estVide(): boolean {
    // La liste est vide ssi tête_ vaut null
    return this.tête_ === null;
  }
}
```

Ajouter en tête d'une valeur `v`

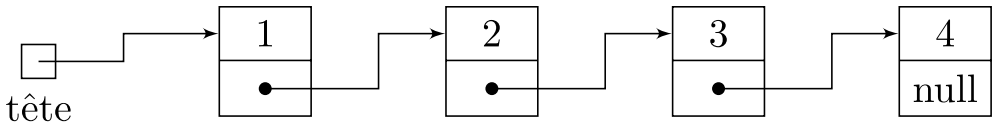
1. Créer le nouveau maillon (structure dynamique) contenant `v` comme donnée et la tête de la liste comme suivant
2. Mettre à jour la tête de la liste

Remarque : l'ordre des actions est **important**

Exemple : Soit la liste



Après ajout de 1 :



Question : Y-a-t'il un problème si la liste est initialement vide ?

Coût : constant

Implémentation en TypeScript

```typescript
class ListeChaînée {
  private tête_: Maillon | null = null; // Initialement la liste est vide

  get tête(): Maillon | null {
    return this.tête_;
  }

  estVide(): boolean {
    // La liste est vide ssi tête_ vaut null
    return this.tête_ === null;
  }

  ajouter(v: number): void {
    let nouveauMaillon = new Maillon(v, this.tête_);
    this.tête_ = nouveauMaillon;
  }
}
```

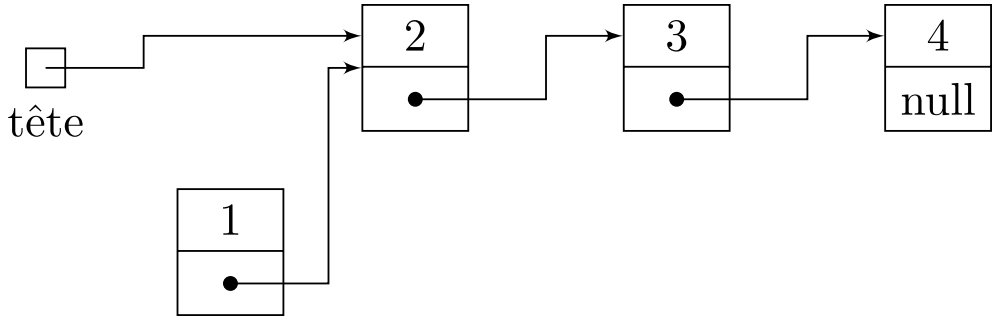
Valeur de l'élément de tête et suite de la liste

- Valeur : retourner la valeur stockée dans le maillon de tête
- Suite : retourner la référence sur le maillon suivant du maillon de tête
- **Question d'implémentation :** que faire si la liste est vide ? erreur, `null` ...
- **Coût :** constant

```
class ListeChaînée {
  // début de la classe

  get valeurTête(): number {
    if (this.estVide()) throw new Error("Liste vide");
    // ! : indique à TS que l'on est sûr que tête_ ne vaut pas null
    return this.tête_!.donnée;
  }

  get suite(): Maillon | null {
    if (this.estVide()) throw new Error("Liste vide");
    return this.tête_!.suivant;
  }
}
```

Retirer l'élément en tête de liste

- Il suffit de modifier la valeur de `tête` sur le suivant du premier maillon



- **Question :** que faire si la liste est vide ? erreur, rien ...

```
class ListeChaînée {
  // début de la classe

  retirer(): void {
    if (this.estVide()) throw new Error("Liste vide");
    this.tête_ = this.tête_!.suivant;
  }
}
```

Coût : constant

Itération sur les éléments d'une liste

- On peut vouloir itérer sur les éléments d'une liste pour :
  - les afficher (*coût linéaire*)
  - rechercher la présence d'un élément (*coût linéaire*)
  - rechercher la position d'un élément $\longrightarrow$ référence vers un maillon (*coût linéaire*)
- Le parcours se fait en passant d'un maillon à son maillon suivant (*problème qui peut se définir récursivement*)
- Définition d'une classe *itérateur* permettant entre autre :
  - avancer (*coût constante*) / reculer (*coût constant*)
  - référence maillon courant / valeur courante (*coût constant*)
  - est-ce la fin ? le début ? (*coût constant*)
  - Ajouter avant / après (*coût constant*)
  - retirer le maillon correspondant... (*coût constant*)
- **Remarque :** un itérateur $\approx$ référence sur un pointeur

Base d'une classe itérateur pour ListeChaînée

```
class ItérateurListeChaînée {
  private liste_: ListeChaînée;
  private ptr_: Maillon | null;

  constructor(liste: ListeChaînée) {
    this.liste_ = liste;
    this.ptr_ = liste.tête;
  }

  get courant(): Maillon | null {
    return this.ptr_;
  }

  get valeurCourante(): number {
    if (this.fin()) throw new Error("Fin de liste");
    return this.ptr_!.donnée;
  }

  fin(): boolean {
    return this.ptr_ === null;
  }

  debut(): boolean {
    return this.ptr_ === this.liste_.tête;
  }

  avancer(): void {
    if (this.fin()) throw new Error("Fin de liste");
    this.ptr_ = this.ptr_!.suivant;
  }
}
```

Remarque : faire reculer l'itérateur n'est pas simple avec une liste simplement chaînée

Ajouter un élément après

1. Créer le nouveau maillon
2. Mettre à jour le suivant du nouveau maillon vers le suivant de l'itérateur
3. Mettre à jour le suivant de l'itérateur vers le nouveau maillon

Attention : l'ordre des actions est important.

```
ajouterAprès(v: number): void {
  if (this.fin()) throw new Error("Fin de liste");
  let nouveauMaillon = new Maillon(v, this.ptr_!.suivant);
  this.ptr_!.suivant = nouveauMaillon;
}
```

Remarque : ajouter avant n'est pas simple avec une liste simplement chaînée.

Rechercher un élément dans la liste et mettre-à-jour l'itérateur

- Déplace l'itérateur jusqu'à la première occurence de la valeur à partir de la position actuellement pointée
- si la valeur n'est pas présente, l'itérateur pointe `null`

```
class ItérateurListeChaînée {
  // début de la classe

  recherche(v: number): void {
    let trouvé = false;
    while (!this.fin() && !trouvé)
      if (this.ptr_!.donnée === v) trouvé = true;
      else this.avancer();
  }
}
```

Méthode de recherche dans `ListeChaînée` avec un itérateur

- On peut utiliser les itérateurs dans la classe `ListeChainée`
- Cela permet d'écrire un code qui utilise l'interface fourni par l'itérateur
- Cela permet un code plus robuste

```
class ListeChaînée {
  // début de la classe

  recherche(v: number): ItérateurListeChaînée {
    let it = new ItérateurListeChaînée(this);
    it.recherche(v);
    return it;
  }
}
```

Méthode de recherche dans `ListeChaînée` sans itérateur

On peut aussi utiliser directement les références mais cela est moins robuste.

```
class ListeChaînée {
  // début de la classe

  rechercheBis(v: number): Maillon | null {
    let aux = this.tête_;
    let trouvé = false;

    while (aux != null && !trouvé)
      if (aux.donnée === v) trouvé = true;
      else aux = aux.suivant;

    return aux;
  }
}
```

Fonction d'affichage d'une liste chaînée

```
function affichage(liste: ListeChaînée): void {
  let it = new ItérateurListeChaînée(liste);

  while (!it.fin()) {
    console.log(it.valeurCourante);
    it.avancer();
  }
}
```

Remarque : dans une approche objet, on ferait plutôt une méthode de `ListeChaînée`

```
affichage(): void {
  let it = new ItérateurListeChaînée(this);

  while (!it.fin()) {
    console.log(it.valeurCourante);
    it.avancer();
  }
}
```